

Bellman Operators: State-Value and Action-Value Functions

Reinforcement Learning

Sai Sampath Kedari

sampath@umich.edu

Contents

1	Setup and the Value Function	2
2	The Bellman Expectation Operator T_π	2
2.1	Definition	2
2.2	What the Operator is Doing	3
3	The Bellman Optimality Operator T	4
3.1	From Policy Evaluation to Optimal Control	4
3.2	Definition	4
3.3	What the Operator is Doing	4
4	Comparison: T_π vs T	5
5	Bellman Operators for Action-Value Functions	6
5.1	Why We Need Q -Space Operators	6
5.2	Deriving the Q -Operators from the V -Operators	6
5.3	Bellman Expectation Operator for Q	7
5.4	Bellman Optimality Operator for Q	8
5.5	The Full Picture: All Four Operators	9
6	From Operators to Algorithms: The Full Spectrum	9
6.1	Model-Free Prediction: T_π on V	9
6.2	Model-Free Control: T_π and T on Q	12
6.3	The Unified Picture	14

1 Setup and the Value Function

We work throughout with an infinite-horizon discounted MDP with finite state space $\mathcal{S}$, finite action sets $\mathcal{A}(s)$ for each $s \in \mathcal{S}$, transition kernel $p(s' \mid s, a)$, bounded expected reward

$$r(s, a) := \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a],$$

and discount factor $0 \leq \gamma < 1$.

For a fixed stationary policy π , the state-value function is the expected discounted return starting from state s and following π forever:

$$v_\pi(s) := \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right].$$

We derived in an earlier report that v_π satisfies the Bellman consistency equation

$$v_\pi(s) = \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) \left[r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' \mid s, a) v_\pi(s') \right] \quad \forall s \in \mathcal{S}.$$

The right-hand side of this equation is a map from functions to functions. This report is about naming that map, understanding what it computes, and extending it to the optimality setting.

2 The Bellman Expectation Operator T_π

2.1 Definition

Definition 2.1 (Bellman Expectation Operator). *For a stationary policy π , the Bellman expectation operator $T_\pi : \mathcal{V} \rightarrow \mathcal{V}$ is defined by*

$$(T_\pi v)(s) := \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) \left[r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' \mid s, a) v(s') \right] \quad \forall s \in \mathcal{S},$$

where $\mathcal{V} = \{v : \mathcal{S} \rightarrow \mathbb{R} : \|v\|_\infty < \infty\}$ is the Banach space of bounded functions on $\mathcal{S}$ under the sup norm $\|v\|_\infty = \sup_{s \in \mathcal{S}} |v(s)|$.

Writing $r_\pi(s) := \sum_a \pi(a \mid s) r(s, a)$ for the policy-weighted expected reward and $P_\pi(s' \mid s) := \sum_a \pi(a \mid s) p(s' \mid s, a)$ for the policy-induced transition kernel, the operator takes the compact matrix form

$$T_\pi v = r_\pi + \gamma P_\pi v.$$

This is an affine map on $\mathcal{V}$: a fixed shift r_π (the immediate reward) plus a discounted linear part $\gamma P_\pi v$ (the expected next-state value). The Bellman consistency equation is now simply the fixed-point equation

$$v_\pi = T_\pi v_\pi.$$

v_π is the unique element of $\mathcal{V}$ that T_π maps to itself. This is the object we are trying to compute when we run policy evaluation.

2.2 What the Operator is Doing

We want to compute $v_{k+1}(s) = (T_\pi v_k)(s)$ for a given state s . Start there and build up the computation naturally.

$$v_{k+1}(s) = (T_\pi v_k)(s) = \sum_{a \in \mathcal{A}(s)} \pi(a | s) \left[r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, a) v_k(s') \right].$$

You are at state s . You want to figure out how good this state is. You do this by looking exactly one step into the future.

Take an action. From state s , pick some action $a \in \mathcal{A}(s)$. This action produces an immediate expected reward $r(s, a)$ and triggers the transition dynamics.

The dynamics land you somewhere. From (s, a) , the environment transitions to some next state s' with probability $p(s' | s, a)$. Many landing states are possible. From each s' , instead of continuing to roll out the full infinite future, use the current estimate $v_k(s')$ to represent everything from s' onward. This is the bootstrap: one real step forward, then trust the current estimate for the rest.

Average over where you could land. You do not know which s' you will end up in, so take the expectation over all possible next states under $p(s' | s, a)$:

$$r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, a) v_k(s').$$

This quantity is the value of committing to action a from state s , given the current estimate v_k of the future.

Average over which action you take. You do not commit to a single action either. Policy $\pi(\cdot | s)$ is a distribution over actions, so average over all feasible $a \in \mathcal{A}(s)$ with weights $\pi(a | s)$:

$$v_{k+1}(s) = \sum_{a \in \mathcal{A}(s)} \pi(a | s) \left[r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, a) v_k(s') \right].$$

This is the full computation: one step of exact expected dynamics under both π and p , then bootstrap with v_k . No trajectories are simulated. No dice are rolled. Both the action and the next state are averaged over their true distributions.

T_π uses the true distributions $\pi(a | s)$ and $p(s' | s, a)$ to take one exact expected step, then hands off to v_k . Every model-free algorithm in the TD and MC family is doing the same computation, but replacing one or both of these averages with samples drawn from the real environment. In TD(0), both the action a and the next state s' come from a single observed transition: the dice are rolled once and the result is used directly. In n -step TD, the dice are rolled n times before the bootstrap kicks in. In Monte Carlo, the dice are rolled all the way to episode termination and there is no bootstrap at all. The operator T_π is the ideal full-average version. Every algorithm from TD(0) to Monte Carlo is what you get when you replace those averages with samples of varying depth.

3 The Bellman Optimality Operator T

3.1 From Policy Evaluation to Optimal Control

The Bellman expectation operator T_π is tied to a specific policy π . Hand it a different policy π' and you get a different operator $T_{\pi'}$ with a different fixed point $v_{\pi'}$. Policy evaluation answers the question: how good is the policy I gave you?

Control asks a different question: what is the best any policy can do? Define the optimal value function as

$$v^*(s) := \max_{\pi} v_{\pi}(s) \quad \forall s \in \mathcal{S}.$$

Now ask: what equation must v^* satisfy? The key observation is that an optimal policy must be greedy with respect to v^* at every state. If it were not greedy at some state s , you could deviate for that one step, collect a higher immediate value, and then continue optimally, which would contradict the assumption that v^* is the value of the optimal policy. This argument forces the optimal policy to satisfy a max at every state, and substituting back gives

$$v^*(s) = \max_{a \in \mathcal{A}(s)} \left[r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, a) v^*(s') \right] \quad \forall s \in \mathcal{S}.$$

This is the Bellman optimality equation. The max over actions replaces the policy-weighted $\sum_a \pi(a | s)$: instead of averaging over actions under a fixed policy, we select the single best action available.

3.2 Definition

Definition 3.1 (Bellman Optimality Operator). *The Bellman optimality operator $T : \mathcal{V} \rightarrow \mathcal{V}$ is defined by*

$$(Tv)(s) := \max_{a \in \mathcal{A}(s)} \left[r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, a) v(s') \right] \quad \forall s \in \mathcal{S}.$$

The Bellman optimality equation is the fixed-point equation $v^* = Tv^*$. The optimal value function is the unique element of $\mathcal{V}$ that T maps to itself, and it can be found by iterating T from any starting point, as proved in the contraction report.

3.3 What the Operator is Doing

We want to compute $v_{k+1}(s) = (Tv_k)(s)$ for a given state s .

$$v_{k+1}(s) = (Tv_k)(s) = \max_{a \in \mathcal{A}(s)} \left[r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, a) v_k(s') \right].$$

You are at state s . You want to know the best possible value you can achieve from here. You look one step into the future, exactly as in T_π .

Take an action. From state s , pick some action $a \in \mathcal{A}(s)$. This gives immediate expected reward $r(s, a)$ and triggers the dynamics.

The dynamics land you somewhere. From (s, a) , the environment transitions to s' with probability $p(s' | s, a)$. Bootstrap with $v_k(s')$ from each landing state instead of rolling out the full future. Average over all possible next states:

$$r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, a) v_k(s').$$

This is the value of committing to action a from s under the current estimate v_k . It is the identical expression as in T_π .

Now take the best action, not the average. This is the only place where T differs from T_π . Instead of averaging over actions under a fixed policy, take the maximum over all feasible actions. You are asking: given what v_k says about the future, which action is worth the most right now?

$$v_{k+1}(s) = \max_{a \in \mathcal{A}(s)} \left[r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, a) v_k(s') \right].$$

The dynamics side is identical to T_π : one step of true averaged dynamics under p , then bootstrap with v_k . The only change is on the action side: $\sum_a \pi(a | s)$ becomes $\max_a$.

T_π asks: if I follow policy π from s , what is one averaged step of dynamics worth? T asks: if I act as greedily as possible from s according to my current estimate v_k , what is one averaged step of dynamics worth? Both operators average over next states under the true dynamics p . Both bootstrap with v_k . The entire difference between policy evaluation and value iteration is a single character: $\sum_a \pi(a | s)$ versus $\max_a$.

4 Comparison: T_π vs T

	Bellman Expectation T_π	Bellman Optimality T
Action selection	Average under $\pi(a s)$	$\max_{a \in \mathcal{A}(s)}$
Dynamics	One step of $p(s' s, a)$	One step of $p(s' s, a)$
Bootstrap	$v_k(s')$	$v_k(s')$
Fixed point	v_π (value of π)	v^* (optimal value)
Contraction modulus	γ	γ
Iteration	Policy evaluation	Value iteration
Origin	Derived from definition of v_π	Derived from optimality of greedy policy

Table 1: Comparison of the two Bellman operators on state-value functions.

Both operators are γ -contractions on the same Banach space $(\mathcal{V}, \|\cdot\|_\infty)$, proved in dedicated earlier reports. Both are solved by fixed-point iteration from any starting point. Both take exactly one step

of real averaged dynamics and bootstrap with the current estimate. The only structural difference is whether the action is averaged under a policy or selected greedily.

5 Bellman Operators for Action-Value Functions

5.1 Why We Need Q -Space Operators

The V -space operators T_π and T compute values indexed by states alone. For model-free control this is insufficient: as established in the Monte Carlo control reports, greedy improvement over v_π requires the transition model $p(s' | s, a)$ to rank actions. The action-value function $q_\pi(s, a)$ resolves this by conditioning on the action upfront, so that greedy improvement reduces to a direct arg max over stored values.

We therefore need T_π and T acting on $\mathcal{Q}$, the space of bounded functions on $\mathcal{S} \times \mathcal{A}$. These operators are not new mathematics. They are the V -space operators with one substitution applied.

5.2 Deriving the Q -Operators from the V -Operators

From the policy evaluation report, the action-value function satisfies

$$q_\pi(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, a) v_\pi(s').$$

The right-hand side still contains $v_\pi(s')$, a state-value. To close the equation in terms of Q alone, substitute the definition of v_π in terms of q_π .

For T_π . Substitute $v_\pi(s') = \sum_{a'} \pi(a' | s') q_\pi(s', a')$:

$$q_\pi(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, a) \sum_{a' \in \mathcal{A}(s')} \pi(a' | s') q_\pi(s', a').$$

The right-hand side is the Bellman expectation operator for Q applied to q_π .

For T . Substitute $v^*(s') = \max_{a'} q^*(s', a')$:

$$q^*(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, a) \max_{a' \in \mathcal{A}(s')} q^*(s', a').$$

The right-hand side is the Bellman optimality operator for Q applied to q^* .

Both derivations follow the same two-step pattern:

$$q(s, a) = r(s, a) + \gamma \sum_{s'} p(s' | s, a) v(s') \xrightarrow{\text{substitute } v(s')} Q\text{-operator}.$$

For T_π : $v_\pi(s') = \sum_{a'} \pi(a' | s') q_\pi(s', a')$. For T : $v^*(s') = \max_{a'} q^*(s', a')$. Nothing else changes. The Q -operators inherit everything from the V -operators through this single substitution.

5.3 Bellman Expectation Operator for Q

Definition 5.1 (Bellman Expectation Operator on $\mathcal{Q}$). Let $\mathcal{Q} = \{Q : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R} : \|Q\|_\infty < \infty\}$ under the sup norm $\|Q\|_\infty = \sup_{s \in \mathcal{S}, a \in \mathcal{A}(s)} |Q(s, a)|$. For a stationary policy π , the Bellman expectation operator $T_\pi : \mathcal{Q} \rightarrow \mathcal{Q}$ is defined by

$$(T_\pi Q)(s, a) := r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, a) \sum_{a' \in \mathcal{A}(s')} \pi(a' | s') Q(s', a').$$

The fixed-point equation is $q_\pi = T_\pi q_\pi$. The action-value function q_π is the unique element of $\mathcal{Q}$ that T_π maps to itself. Since both $p(\cdot | s, a)$ and $\pi(\cdot | s')$ are probability distributions, neither increases the sup norm. The only scaling factor is $\gamma < 1$, giving contraction modulus γ . The full proof is in the contraction report.

What the operator is doing. We want to compute $Q_{k+1}(s, a) = (T_\pi Q_k)(s, a)$.

$$Q_{k+1}(s, a) = (T_\pi Q_k)(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, a) \sum_{a' \in \mathcal{A}(s')} \pi(a' | s') Q_k(s', a').$$

You are at (s, a) . The action is already committed as part of the input. Unlike the V -space operator, there is no averaging over the first action. The computation starts from the pair (s, a) directly.

Collect the immediate reward. Action a from state s yields immediate expected reward $r(s, a)$.

Deploy one step of real dynamics. The environment transitions to s' with probability $p(s' | s, a)$. Average over all possible next states:

$$r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, a) (\dots).$$

This is one real step of dynamics, identical to the V -space operator.

You have landed in s' . Average over the next action under π . Policy $\pi(\cdot | s')$ selects the next action a' with probability $\pi(a' | s')$. Average over all feasible $a' \in \mathcal{A}(s')$:

$$\sum_{a' \in \mathcal{A}(s')} \pi(a' | s') Q_k(s', a').$$

This quantity is exactly $v_k(s') = \sum_{a'} \pi(a' | s') Q_k(s', a')$: the current estimate of the state value at s' , reconstructed from Q -values by averaging under π .

Bootstrap. Use the current estimate $Q_k(s', a')$ for everything beyond the pair (s', a') .

In the V -space operator T_π , you start at state s and immediately average over the first action under π , then step. In the Q -space operator T_π , the first action is fixed as part of the input (s, a) , so you step first, land in s' , and then average over the next action under π . One real dynamics step in both cases. The policy averaging just happens at a different point: before the step for V , after the step for Q .

The parallel between T_π on V and T_π on Q is made explicit by writing the two update equations side by side:

$$v_{k+1}(s) = \underbrace{\sum_{a \in \mathcal{A}(s)} \pi(a | s)}_{\text{average first action}} \left[r(s, a) + \gamma \sum_{s'} p(s' | s, a) v_k(s') \right]$$

$$Q_{k+1}(s, a) = r(s, a) + \gamma \sum_{s'} p(s' | s, a) \underbrace{\sum_{a' \in \mathcal{A}(s')} \pi(a' | s') Q_k(s', a')}_{\text{average next action}}$$

In T_π on V : average over the current action first, then step, then bootstrap. In T_π on Q : step first (action already fixed), land in s' , then average over the next action, then bootstrap.

5.4 Bellman Optimality Operator for Q

Definition 5.2 (Bellman Optimality Operator on Q). *The Bellman optimality operator $T : \mathcal{Q} \rightarrow \mathcal{Q}$ is defined by*

$$(TQ)(s, a) := r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, a) \max_{a' \in \mathcal{A}(s')} Q(s', a').$$

The fixed-point equation is $q^* = Tq^*$. The optimal action-value function is the unique element of $\mathcal{Q}$ that T maps to itself. The contraction proof uses the max-is-1-Lipschitz lemma exactly as in the V -space case, with the index set $\mathcal{S} \times \mathcal{A}$ replacing $\mathcal{S}$.

What the operator is doing. We want to compute $Q_{k+1}(s, a) = (TQ_k)(s, a)$.

$$Q_{k+1}(s, a) = (TQ_k)(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, a) \max_{a' \in \mathcal{A}(s')} Q_k(s', a').$$

You are at (s, a) . The action is already committed as part of the input. Identical starting point to T_π on Q .

Collect the immediate reward. Action a from state s yields immediate expected reward $r(s, a)$.

Deploy one step of real dynamics. Transition to s' with probability $p(s' | s, a)$. Average over all possible next states:

$$r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, a) (\dots).$$

Identical to T_π on Q up to this point.

You have landed in s' . Take the best next action, not the average. This is the only place where T on Q differs from T_π on Q . Instead of averaging over the next action under π , take the maximum over all feasible a' :

$$\max_{a' \in \mathcal{A}(s')} Q_k(s', a').$$

This is $v_k^*(s') = \max_{a'} Q_k(s', a')$: the current estimate of the optimal state value at s' , taken as the best available action-value.

Bootstrap. Use $Q_k(s', a')$ for everything beyond (s', a') .

The Q -space operators T_π and T differ in exactly the same way as their V -space counterparts. After taking one real dynamics step and landing in s' , T_π on Q asks: what does policy π do from here on average? T on Q asks: what is the best action available here? The difference is $\sum_{a'} \pi(a' | s')$ versus $\max_{a'}$, appearing at s' after the step. Everything else, the starting pair, the dynamics step, the bootstrap, is identical.

5.5 The Full Picture: All Four Operators

Operator	Space	First action	At next state s'	Fixed point
T_π on V	$\mathcal{V}$	Average $\sum_a \pi(a s)$	Bootstrap $v_k(s')$	v_π
T on V	$\mathcal{V}$	$\max_{a \in \mathcal{A}(s)}$	Bootstrap $v_k(s')$	v^*
T_π on Q	$\mathcal{Q}$	Fixed (input a)	Average $\sum_{a'} \pi(a' s') Q_k(s', a')$	q_π
T on Q	$\mathcal{Q}$	Fixed (input a)	$\max_{a'} Q_k(s', a')$	q^*

Table 2: All four Bellman operators. Every operator takes one step of real dynamics under $p(s' | s, a)$ and bootstraps with the current estimate. The differences are: which space the operator acts on, whether the first action is averaged or fixed, and whether the next action at s' is averaged under π or maximised over.

The four operators are one idea in four forms. The idea is always: take one step of real expected dynamics, handle the action choice (average or max), and bootstrap. The space (V or Q) determines whether the first action is part of the input or averaged over. The operator (T_π or T) determines whether actions are averaged under a policy or selected greedily. Every algorithm in the TD and MC family, and every DP algorithm, is a version of one of these four operators with the expectations replaced by samples to varying depth.

6 From Operators to Algorithms: The Full Spectrum

6.1 Model-Free Prediction: T_π on V

The goal is to estimate v_π for a fixed policy π without access to the model. The operator defines the exact computation we want to perform:

$$(T_\pi v_k)(s) = \sum_{a \in \mathcal{A}(s)} \pi(a | s) \left[r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, a) v_k(s') \right].$$

This requires two things we do not have in the model-free setting: the ability to evaluate $r(s, a)$ for any action a , and the ability to evaluate $p(s' | s, a)$ for any next state s' . We cannot compute these expectations.

What we can do is the opposite: we can *sample* from the distributions, even though we cannot evaluate them. Executing the policy in the real environment draws $A_t \sim \pi(\cdot | S_t)$ and then the

world draws $S_{t+1} \sim p(\cdot \mid S_t, A_t)$ and hands us back R_{t+1} . The environment is a black box that samples from p for us, even though we never see p itself. This is the opposite of importance sampling, where we can evaluate the target distribution but cannot sample from it. Here we can sample but cannot evaluate.

The key design choice that separates every prediction algorithm is: after sampling from the environment, how many steps do we take before we stop sampling and bootstrap with our current estimate?

TD(0): replace both averages with one sample each. Start with the operator and replace each expectation with a single draw.

1. **Sample an action.** Instead of averaging $\sum_a \pi(a \mid S_t)[\dots]$, draw one action:

$$A_t \sim \pi(\cdot \mid S_t).$$

2. **Sample the next state.** Instead of averaging $\sum_{s'} p(s' \mid S_t, A_t)v_k(s')$, execute A_t in the real world. The environment returns one reward and one next state:

$$R_{t+1}, S_{t+1} \sim p(\cdot \mid S_t, A_t).$$

3. **Bootstrap immediately.** Use the current estimate $\hat{v}(S_{t+1})$ for all value from S_{t+1} onward.

The update is:

$$\hat{v}(S_t) \leftarrow \hat{v}(S_t) + \alpha \left(\underbrace{R_{t+1} + \gamma \hat{v}(S_{t+1})}_{\text{one real step, then bootstrap}} - \hat{v}(S_t) \right).$$

Both averages in T_π have been replaced by single samples. The bootstrap depth is still one step, exactly as in the operator. The cost is bias: $\hat{v}(S_{t+1})$ is not the true $v_\pi(S_{t+1})$.

n -step TD: replace both averages with n samples each. Instead of bootstrapping after one real step, take n real steps first. At each step $k = 1, \dots, n$:

$$A_{t+k-1} \sim \pi(\cdot \mid S_{t+k-1}), \quad R_{t+k}, S_{t+k} \sim p(\cdot \mid S_{t+k-1}, A_{t+k-1}).$$

After n steps of real interaction, bootstrap with $\hat{v}(S_{t+n})$:

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n \hat{v}(S_{t+n}).$$

The update is $\hat{v}(S_t) \leftarrow \hat{v}(S_t) + \alpha(G_t^{(n)} - \hat{v}(S_t))$. Each reward and each state transition is a real observation from the environment. Only the tail beyond step n is approximated by the bootstrap. As n grows, less is approximated and more is observed, so bias decreases. But more random quantities contribute to the target, so variance increases.

Monte Carlo: sample all the way to termination, no bootstrap. Take n to its limit: sample every step until the episode ends at time T .

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-t-1} R_T.$$

No bootstrap at all. Every reward and every transition is a real observation. The target G_t is a single sample of the quantity $v_\pi(S_t)$ is defined as, so $\mathbb{E}_\pi[G_t \mid S_t = s] = v_\pi(s)$ exactly. The estimate is unbiased. The cost is high variance: the target depends on the entire random trajectory from t to T .

The spectrum. The figure below illustrates this progression. Each tree corresponds to one method. Filled circles are sampled action-state transitions (real environment steps). Open circles are sampled states. The square at the bottom of the Monte Carlo tree is the terminal state. The triangle on the non-Monte Carlo trees marks the bootstrap point.

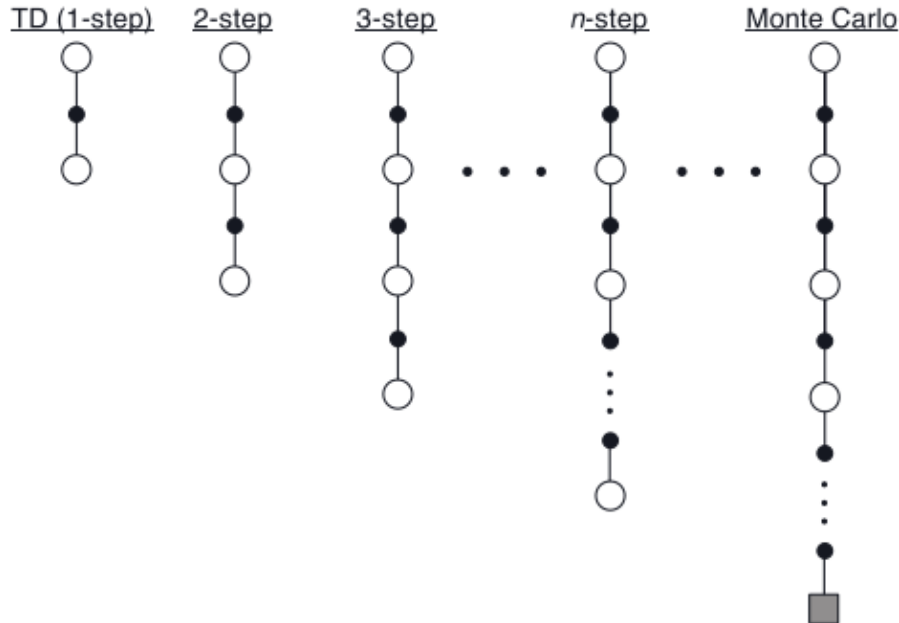


Figure 1: The spectrum from TD(0) to Monte Carlo. TD(1) takes one real step and bootstraps. n -step TD takes n real steps and bootstraps. Monte Carlo samples all the way to episode termination with no bootstrap. Every method is T_π on V with the two expectations replaced by samples of varying depth.

Method	Action samples	Dynamics samples	Bootstrap at	Bias
Operator $T_\pi v$	None (full avg)	None (full avg)	Step 1	Zero
TD(0)	1	1	Step 1	High
n -step TD	n	n	Step n	Decreasing
Monte Carlo	$T - t$	$T - t$	Never	Zero

Table 3: Every prediction method is T_π on V with expectations replaced by samples. The operator uses no samples and has zero bias. Monte Carlo uses all samples and has zero bias via the law of large numbers. TD(0) uses the fewest samples and has the highest bias from bootstrapping.

In importance sampling, you can evaluate the target distribution but cannot sample from it, so you reweight samples from a different distribution. Here the situation is reversed: you cannot evaluate $p(s' | s, a)$ or $r(s, a)$ for arbitrary inputs, but you can sample from them by running the real environment. The environment is an oracle that draws from the unknown distribution on demand. Every model-free algorithm exploits exactly this: replace the intractable expectation with samples drawn by real interaction, and choose how many steps to sample before falling back to the bootstrap.

6.2 Model-Free Control: T_π and T on Q

In control the goal is to find q^* , not merely evaluate a given policy. Working with v_π is ruled out for model-free control because greedy improvement over v_π requires a one-step model rollout to rank actions. The Q -space operator resolves this: the action is already fixed as part of the input, so no model is needed for improvement.

The evaluation operator on Q is:

$$(T_\pi Q_k)(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, a) \sum_{a' \in \mathcal{A}(s')} \pi(a' | s') Q_k(s', a').$$

You are at (s, a) . The first action a is already committed as part of the input, so there is no averaging over the first action. The operator has two remaining sources of averaging: the dynamics $p(s' | s, a)$ over where you land, and the policy $\pi(a' | s')$ over which action you take from there. Both require the model to evaluate exactly. Both are replaced by real environment samples in practice.

SARSA(0): replace both averages with one sample each.

Start with the operator and replace each expectation with a single draw, exactly as in TD(0) for prediction.

1. **You are at (S_t, A_t) .** The current action A_t is already committed. Collect the immediate reward and sample the next state by executing A_t in the real environment:

$$R_{t+1}, S_{t+1} \sim p(\cdot | S_t, A_t).$$

Instead of averaging $\sum_{s'} p(s' | S_t, A_t) (\dots)$, the environment draws S_{t+1} for you.

2. **Sample the next action.** Instead of averaging $\sum_{a'} \pi(a' | S_{t+1}) Q_k(S_{t+1}, a')$, draw one action from the policy at the new state:

$$A_{t+1} \sim \pi(\cdot | S_{t+1}).$$

3. **Bootstrap immediately.** Use the current estimate $Q(S_{t+1}, A_{t+1})$ for all value from (S_{t+1}, A_{t+1}) onward.

The update is:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \right).$$

The quintuple $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$ is the unit of update, which is where the name SARSA comes from. Both averages in the operator are replaced by single samples. Bootstrap depth is one

step, identical to the operator. Policy improvement runs in parallel: after each update, re-derive the ε -greedy policy with respect to the updated Q .

n -step SARSA: replace both averages with n samples each.

Instead of bootstrapping after one real step, take n real steps. At each step $k = 1, \dots, n$, execute the current policy and let the environment step forward:

$$A_{t+k-1} \sim \pi(\cdot \mid S_{t+k-1}), \quad R_{t+k}, S_{t+k} \sim p(\cdot \mid S_{t+k-1}, A_{t+k-1}).$$

After n steps, sample one more action from the policy at the landing state S_{t+n} , then bootstrap:

$$q_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n Q(S_{t+n}, A_{t+n}).$$

The update is $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(q_t^{(n)} - Q(S_t, A_t))$. Every reward and every transition in the n -step window is a real observation. Only the value beyond step n is approximated by the bootstrap $Q(S_{t+n}, A_{t+n})$.

Setting $n = 1$ recovers SARSA(0). As n grows, less is approximated and more is observed: bias decreases and variance increases.

Monte Carlo control: sample all the way to termination, no bootstrap.

Take n to its limit. Sample every action and every transition until the episode ends at time T :

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-t-1} R_T.$$

No bootstrap at all. The target for $Q(S_t, A_t)$ is the full observed return from the pair (S_t, A_t) to termination. Since

$$\mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] = q_\pi(s, a)$$

exactly, the target is unbiased. The cost is high variance: the target depends on the entire trajectory from t to T .

The spectrum.

Method	Dynamics	Policy at s'	Bootstrap	Bias
Operator $T_\pi Q$	Full avg	Full avg	Step 1	Zero
SARSA(0)	1 sample	1 sample	Step 1	High
n -step SARSA	n samples	n samples	Step n	Decreasing
MC control	All steps	All steps	Never	Zero

Table 4: Model-free control as a spectrum. Every method is T_π on Q with the two expectations replaced by samples of varying depth.

The structure of T_π on $\mathcal{Q}$ is: fix the first action, take one dynamics step, average over the next action, bootstrap. SARSA(0) follows the same structure but replaces both averages with real samples. n -step SARSA repeats this sampling n times before bootstrapping. Monte Carlo repeats until termination and never bootstraps. The Q -values make this entire spectrum model-free: greedy improvement requires no model because the action ranking is already encoded in the stored Q -values.

6.3 The Unified Picture

Every algorithm covered in this report, across prediction and control, across TD and Monte Carlo, is the same computation performed at different sampling depths. The operator is always the reference point.

For prediction, the reference is T_π on $\mathcal{V}$:

$$(T_\pi v_k)(s) = \sum_a \pi(a | s) \left[r(s, a) + \gamma \sum_{s'} p(s' | s, a) v_k(s') \right].$$

For control, the reference is T_π on $\mathcal{Q}$:

$$(T_\pi Q_k)(s, a) = r(s, a) + \gamma \sum_{s'} p(s' | s, a) \sum_{a'} \pi(a' | s') Q_k(s', a').$$

Both operators use the true distributions. Both bootstrap with the current estimate after one averaged step. Every model-free algorithm replaces those true averages with samples drawn by real interaction with the environment, and chooses how many steps to sample before the bootstrap.

Method	Space	Operator	Dynamics	Policy	Bootstrap
Iterative policy eval	$\mathcal{V}$	T_π	Full avg	Full avg	Step 1
TD(0)	$\mathcal{V}$	T_π	1 sample	1 sample	Step 1
n -step TD	$\mathcal{V}$	T_π	n samples	n samples	Step n
Monte Carlo pred.	$\mathcal{V}$	T_π	All steps	All steps	Never
SARSA(0)	$\mathcal{Q}$	T_π	1 sample	1 sample	Step 1
n -step SARSA	$\mathcal{Q}$	T_π	n samples	n samples	Step n
Monte Carlo control	$\mathcal{Q}$	T_π	All steps	All steps	Never
Q-Learning	$\mathcal{Q}$	T	1 sample	max	Step 1

Table 5: Every DP and model-free algorithm as an instantiation of one of the four Bellman operators. The top block is prediction ($\mathcal{V}$ -space). The bottom block is control ($\mathcal{Q}$ -space). Within each block the only axis of variation is sampling depth. Q-Learning is the one algorithm that uses T instead of T_π : it replaces the policy average at s' with a max, targeting q^* directly rather than q_π .

Three observations follow from this table.

Prediction and control share the same operator. The move from $\mathcal{V}$ to $\mathcal{Q}$ changes the index set and shifts the policy averaging from before the dynamics step to after it, but the sampling logic is identical. TD(0) for prediction and SARSA(0) for control are the same algorithm in different spaces.

The bootstrap depth is the only free parameter within each row. Given a fixed operator and a fixed space, every algorithm in that row is identical except for how many real steps are taken before the bootstrap. Increasing depth reduces bias and increases variance. This is the fundamental bias-variance tradeoff in model-free RL, and it has nothing to do with the algorithm structure: it is purely a consequence of how many samples replace the true expectation.

Q-Learning is the only algorithm here that uses T instead of T_π . Rather than averaging over the next action under the current policy, it takes the max. This makes it off-policy: the behaviour policy that generates transitions can be anything with full support, while the target is always q^* . SARSA targets q_π for the policy it is following. Q-Learning targets q^* directly.

One step of real dynamics, handle the action (average or max), bootstrap. That is every algorithm in this table. The operator defines what the algorithm is trying to compute. The sampling depth defines how much of that computation uses real observations versus the current estimate. Understanding this structure makes every algorithm in the reports that follow a natural consequence rather than a separate invention.